

디지털 영상 처리

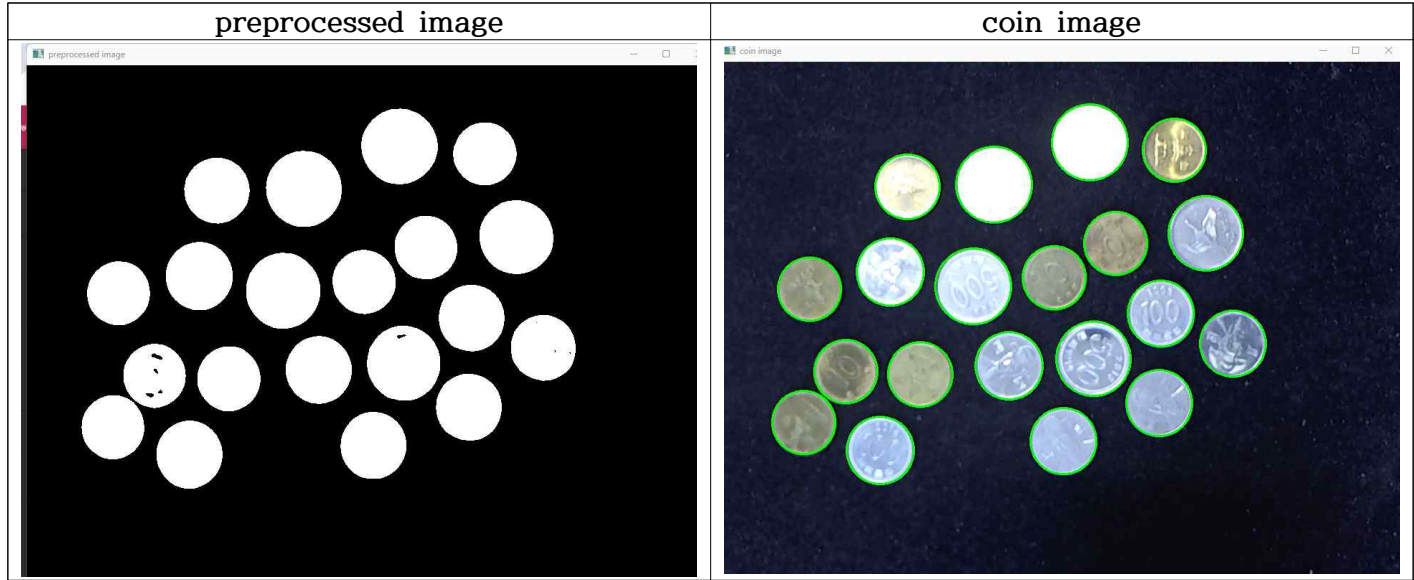
Chap 06 어파인, 원근 변환



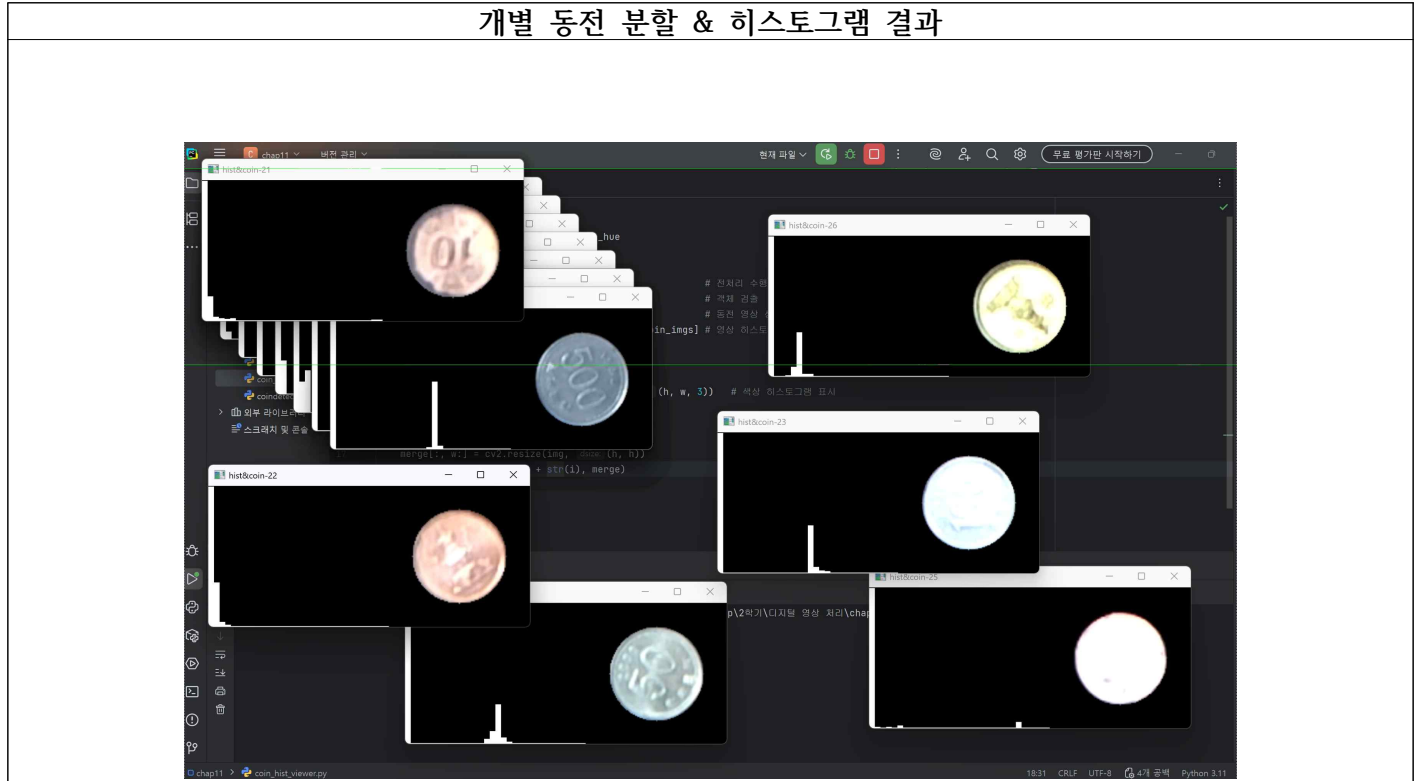
충북대학교
CHUNGBUK NATIONAL UNIVERSITY

날짜	2025.12.02.
이름	권택주
학번	2023042028
학과	지능로봇공학과
담당 교수	황영배 교수님

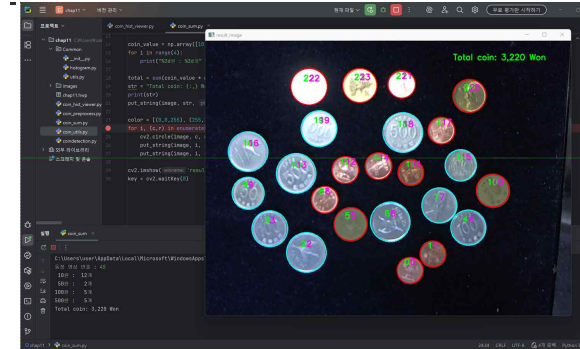
1. 동전 객체 검출 결과



2. 개별 동전 분할 및 히스토그램 결과



3. 동전 계산 프로그램

번호	이미지
1	

The screenshot shows a Windows desktop with a Visual Studio Code editor. The editor has a C++ file named 'main.cpp' open, which contains a program to calculate the sum of numbers from 1 to 100. The code is as follows:

```

#include <iostream>
using namespace std;
int main() {
    int sum = 0;
    for (int i = 1; i <= 100; i++) {
        sum += i;
    }
    cout << sum << endl;
    return 0;
}

```

The terminal window at the bottom shows the output of the program:

```

C:\Users\user> g++ main.cpp
C:\Users\user> ./main.exe
5050

```

The total sum calculated is 5050.

[illegible][illegible]

The screenshot shows a Windows desktop with a VS Code editor and a terminal window. The VS Code editor displays a Python script for K-Means clustering. The terminal window shows the execution of the script, which outputs the total cost, iteration counts, and the final cluster assignments for 15 data points. The data points are visualized as 15 Korean 100-won coins on a dark background, with their assigned cluster numbers (0-4) displayed on them.

Python Script (kmeans.py):

```

import random
import math

def euclidean(x1, y1, x2, y2):
    return math.sqrt((x1-x2)**2 + (y1-y2)**2)

def rand_initialize(k, n):
    for i in range(k):
        print("k: ", i)
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        print(x, y)
        cluster[i] = (x, y)

def init():
    for i in range(k):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        print(x, y)
        cluster[i] = (x, y)

def cost():
    total = 0
    for i in range(n):
        x = data[i][0]
        y = data[i][1]
        for j in range(k):
            dist = euclidean(x, y, cluster[j][0], cluster[j][1])
            if dist < min_dist:
                min_dist = dist
                min_cluster = j
        total += min_dist**2
    return total

def main():
    k = 5
    n = 15
    data = []
    for i in range(n):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        data.append((x, y))
    cluster = []
    for i in range(k):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        cluster.append((x, y))
    cost()
    for i in range(1000):
        cost()
        for j in range(n):
            x = data[j][0]
            y = data[j][1]
            for k in range(k):
                dist = euclidean(x, y, cluster[k][0], cluster[k][1])
                if dist < min_dist:
                    min_dist = dist
                    min_cluster = k
            data[j] = (cluster[min_cluster][0], cluster[min_cluster][1])
        cost()
    print("Total Cost: ", total)
    print("Cluster: ", cluster)
    print("Data: ", data)

```

Terminal Output:

```

k: 0
0.8488888888888889 0.1911111111111111
k: 1
0.1222222222222222 0.8777777777777778
k: 2
0.1222222222222222 0.8777777777777778
k: 3
0.1222222222222222 0.8777777777777778
k: 4
0.1222222222222222 0.8777777777777778
Total Cost: 3.280
Cluster: [(0.8488888888888889, 0.1911111111111111), (0.1222222222222222, 0.8777777777777778), (0.1222222222222222, 0.8777777777777778), (0.1222222222222222, 0.8777777777777778), (0.1222222222222222, 0.8777777777777778)]
Data: [(0.8488888888888889, 0.1911111111111111), (0.1222222222222222, 0.8777777777777778), (0.1222222222222222, 0.8777777777777778), (0.1222222222222222, 0.8777777777777778), (0.1222222222222222, 0.8777777777777778)]

```

Data Points (Coins):

Point ID	Cluster
1	0
2	1
3	2
4	3
5	4
6	0
7	1
8	2
9	3
10	4
11	0
12	1
13	2
14	3
15	4